

Actividad 1 — Diseño del Núcleo de Negocio (Domain Design)

Pronostix — proyecciones estadísticas para apuestas deportivas

Autores: Juan David Sierra, Juan José Palacio, Camilo Soto

Aplicación sobre Django · Transacciones y pasarela de pago propias

1. El Business Case (Nicho de Mercado)

Definición clara (nombre y nicho específico)

Pronostix no es "una app de deportes". Es una **plataforma freemium de proyecciones estadísticas para apuestas deportivas de cualquier deporte y liga** (fútbol, básquetbol, tenis, e-sports, etc.). Para cada evento calcula, con un modelo propio, las probabilidades de TODOS los mercados de análisis (resultado / moneyline, totales over-under, hándicap, anotadores y otros). Es **análisis de entretenimiento e información, NO una casa de apuestas**: solo muestra probabilidades; nunca recibe ni procesa apuestas.

Transaccionalidad (flujo completo de compra / suscripción)

El sistema tiene lógica de negocio transaccional real y de complejidad suficiente para el semestre:

- **Suscripción** a un `Plan` (Free / Premium / Pro) con vigencia, renovación y cancelación.
- **Compra puntual** de un "pase" para desbloquear los mercados premium de un `Evento` específico.
- Todo pago se **liquida por una pasarela construida por nosotros** (`PasarelaPago`): autoriza, captura, evalúa riesgo y reembolsa. No se delega a un procesador externo; ese es el corazón académico de la actividad.

Propuesta de valor (problema que resuelve) y Atributos de Calidad

El análisis estadístico deportivo de nivel profesional (modelos de anotación esperada, ratings de equipo, forma reciente) vive en herramientas caras o cerradas. **Pronostix democratiza ese análisis** y lo entrega al apostador y al aficionado en un formato claro, para cualquier deporte. De la propuesta se derivan los atributos de calidad (Slide 5):

Atributo de calidad	Por qué lo exige el negocio
Consistencia transaccional	Un cobro nunca puede quedar a medias: una <code>Orden</code> está pagada o no lo está.
Seguridad	Se manejan datos de pago; el número de tarjeta nunca se guarda en claro (se tokeniza).
Auditabilidad / Trazabilidad	Cada <code>Prediccion</code> se guarda con timestamp y versión de modelo; cada <code>Transaccion</code> deja rastro.

Atributo de calidad	Por qué lo exige el negocio
Extensibilidad multideporte	Agregar un deporte o liga nueva no debe reescribir el núcleo: el modelo se calibra por Deporte.
Rendimiento	El recálculo del modelo y las consultas de mercados deben responder rápido.

2. El Modelo de Dominio (Diagrama de Clases)

El dominio se organiza en **tres módulos (bounded contexts)**: **Acceso** (identidad y derecho de uso), **Monetización** (ventas y pasarela propia) y **Predicciones** (el producto). Son **15 clases** (el mínimo pedido era 5–7). Cada clase incluye atributos de negocio y responsabilidades (métodos).

Módulo Acceso

Usuario

nombre, email, passwordHash, fechaRegistro, estado
registrar(), autenticar(): bool, tienePremium(): bool, puedeVer(mercado): bool

Plan

nombre, precioMensual, mercadosIncluidos, limiteConsultasDia
incluyeMercado(tipo): bool, precioAnual(): Dinero, esGratis(): bool

Suscripcion

fechaInicio, fechaFin, estado, renovacionAuto
estaVigente(): bool, renovar(), cancelar(), diasRestantes(): int

Módulo Monetización (pasarela propia)

Orden

fechaCreacion, estado, moneda
calcularTotal(): Dinero, agregarLinea(item), confirmarPago(), reembolsar(), estaPagada(): bool

LineaOrden

concepto, descripcion, precioUnitario, cantidad
subtotal(): Dinero

Transaccion

monto, moneda, estado, referencia, fechaProcesado
aprobar(), rechazar(motivo), estaAprobada(): bool

MetodoPago

tipo, tokenTarjeta, ultimos4, expiracion
tokenizar(), estaVigente(): bool, enmascarar(): String

PasarelaPago

nombre, claveComercio
autorizar(tx): bool, capturar(tx), reembolsar(tx), evaluarRiesgo(tx): Score

Factura

numero, fechaEmision, subtotal, impuestos

generar(), total(): Dinero, enNumeros(): String

Módulo Predicciones (el producto)

Competicion

*nombre, deporte, temporada, pais
equipos(): List, calendario(): List, estaActiva(): bool*

Equipo

*nombre, pais, deporte, ranking
promedioAFavor(): float, promedioEnContra(): float*

Evento

*fecha, fase, sede, marcadorLocal, marcadorVisitante, estado
estaEnVivo(): bool, haFinalizado(): bool, resultado(): Resultado, esPremium(): bool*

Mercado

*tipo, esPremium
opciones(): List, prediccionVigente(): Prediccion*

Prediccion

*probabilidades, valorEsperado, fechaCalculo, versionModelo
probabilidadDe(opcion): float, masProbable(): Opcion, fueAcertada(resultado): bool*

ModeloPredictivo

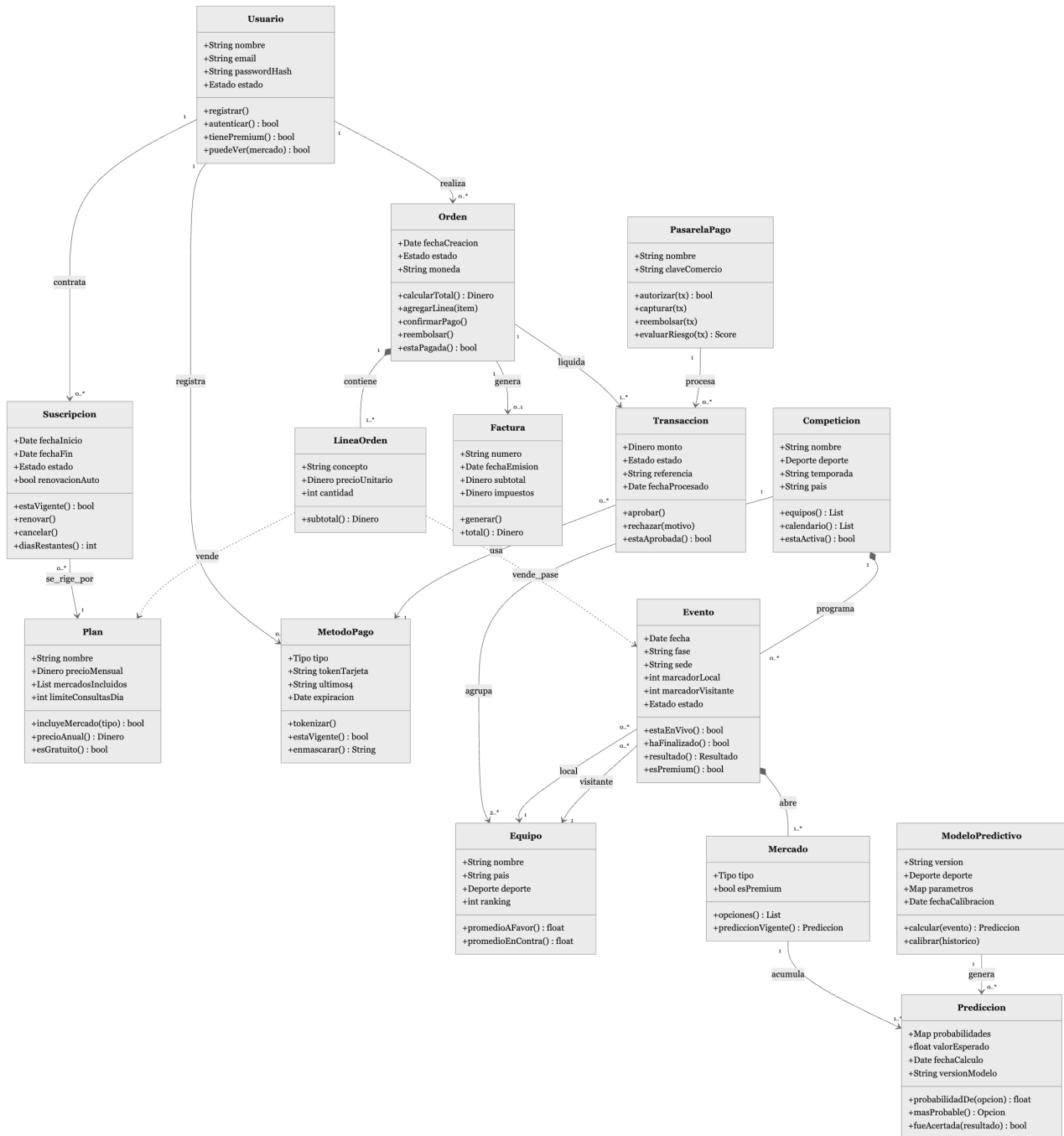
*version, deporte, parametros, fechaCalibracion
calcular(evento): Prediccion, calibrar(historico), version(): String*

Relaciones y multiplicidades

Origen	Relación	Destino	Mult.	Semántica
Usuario	asociación	Suscripcion	1 : 0..*	contrata varias suscripciones en el tiempo
Suscripcion	asociación	Plan	0..* : 1	cada suscripción se rige por un plan
Usuario	asociación	Orden	1 : 0..*	realiza muchas órdenes
Usuario	asociación	MetodoPago	1 : 0..*	registra varios métodos de pago
Orden	composición	LineaOrden	1 : 1..*	la orden se compone de líneas
Orden	asociación	Transaccion	1 : 1..*	se liquida con uno o más intentos de cobro
Orden	asociación	Factura	1 : 0..1	una orden pagada genera a lo sumo una factura
Transaccion	asociación	MetodoPago	0..* : 1	cada transacción usa un método de pago

Origen	Relación	Destino	Mult.	Semántica
PasarelaPago	asociación	Transaccion	1 : 0..*	la pasarela procesa todas las transacciones
LineaOrden	dependencia	Plan	..>	una línea puede vender un plan (cruza módulo)
LineaOrden	dependencia	Evento	..>	una línea puede vender un pase premium (cruza módulo)
Competicion	composición	Evento	1 : 0..*	la competición programa sus eventos
Competicion	asociación	Equipo	1 : 2..*	agrupa a los equipos participantes
Evento	asociación (local)	Equipo	0..* : 1	equipo local
Evento	asociación (visitante)	Equipo	0..* : 1	equipo visitante
Evento	composición	Mercado	1 : 1..*	el evento abre sus mercados
Mercado	asociación	Prediccion	1 : 1..*	histórico de predicciones con timestamp
ModeloPredictivo	asociación	Prediccion	1 : 0..*	el modelo genera predicciones

Diagrama de Clases



Rombo lleno = composición | flecha con multiplicidad = asociación | línea punteada = dependencia (hilo entre módulos).

3. El "Strangler Test" (Visión a Futuro)

Modularidad inicial

Las 15 clases se agrupan en tres módulos con responsabilidad única: **Acceso** (Usuario, Plan, Suscripcion), **Monetización** (Orden, LineaOrden, Transaccion, MetodoPago, PasarelaPago, Factura) y **Predicciones** (Competicion, Equipo, Evento, Mercado, Prediccion, ModeloPredictivo). Se puede señalar qué clases son "Ventas/Monetización" y cuáles son "Producto/Predicciones" sin ambigüedad.

Bajo acoplamiento (sin God Class)

El conocimiento está distribuido; ninguna clase "hace de todo":

- El **cobro** vive en PasarelaPago, separado de la Orden (la orden no sabe cómo se autoriza una tarjeta; solo pide "confírmame el pago").
- El **cálculo** vive en ModeloPredictivo, separado de la Prediccion (la predicción es el resultado inmutable; el modelo la produce).
- Los dos módulos grandes se tocan por un **único hilo delgado**: LineaOrden depende de Plan o de Evento (dependencia, no composición). Se podría "estrangular" y extraer el módulo de Predicciones a otro servicio cortando solo esa dependencia.

Independencia de framework

El diagrama y todos los nombres son de dominio de negocio: Orden.confirmarPago(), Modelo.calcular(), Suscripcion.estaVigente(). No aparece HttpRequest, models.py, QuerySet ni nada de Django. Se entiende sin saber el framework de implementación.

Cobertura del checklist

	Ítem	Dónde se cumple
X	Definición clara (nombre y nicho)	§1 — Pronostix, proyecciones multideporte
X	Transaccionalidad	§1 — suscripción y pase, pasarela propia
X	Propuesta de valor y atributos de calidad	§1 — tabla Slide 5
X	Entidades core (5–7)	§2 — 15 clases en 3 módulos
X	Atributos de negocio	§2 — datos mínimos por clase
X	Responsabilidades (métodos)	§2 — comportamientos por clase
X	Relaciones y multiplicidades	§2 — tabla y diagrama
X	Modularidad inicial	§3 — 3 bounded contexts
X	Bajo acoplamiento	§3 — Pasarela y Modelo aislados
X	Independencia de framework	§3 — dominio puro, sin Django